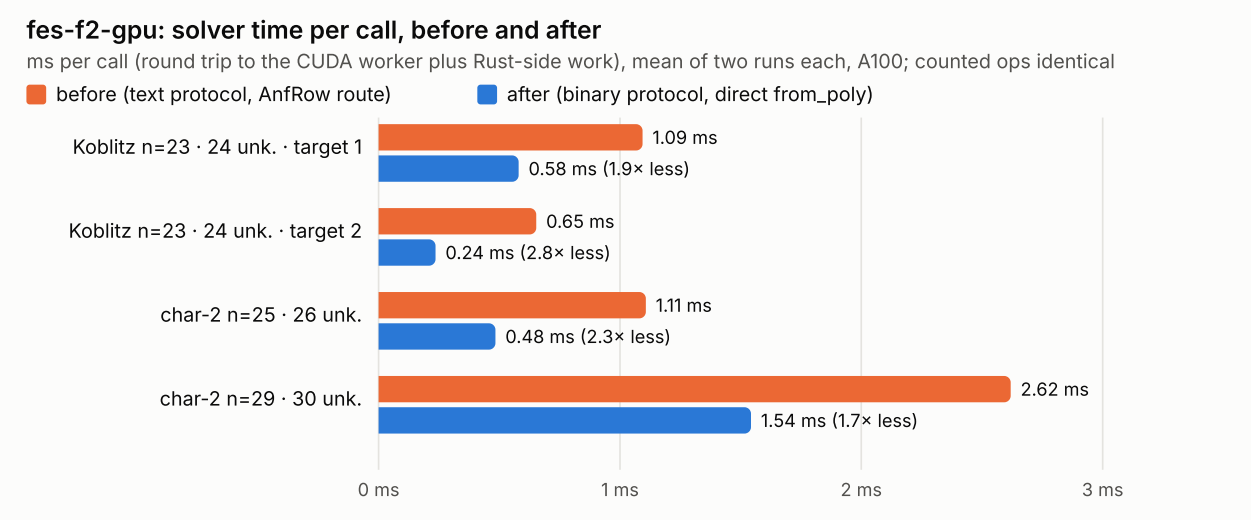
fes-f2-gpu: per-call overhead cut

User request (2026-11-13, “Yes onwards”): cut the per-call overhead of `fes-f2-gpu`, which `research/ic_gpu_e2e_20261113` measured at 1.0–3.6 ms per call against 0.1–1.3 ms of kernel time.

Standing. `[pcc:unverified]`: one pod, measured by the session that wrote the code, no validator. Counted ops, calls and hit rate are identical in both arms, so counted S does not move; the wall columns are a practicality note (`AGENTS.md` §6). Engineering class, no scoreboard row.



Solver time per call, before and after, [pcc:unverified]

Where the overhead was

From the first end-to-end run’s JSON, about half of each call was outside the worker round trip, Rust-side: 0.44–0.66 ms per call at 24–26 unknowns, 1.75 ms at 30. A local probe on degree-29 descent systems (30 unknowns, 29 equations, 3542 terms; ignored test `fes_gpu_rust_side_overhead_probe`) put the engine body around the search at 0.569 ms, of which `AnfRow::from_poly` was 0.421 ms: `Mask` is `u128`, and every term scanned 128 bit positions and allocated a `Vec`.

Changes (commit `16e7903d26`)

- `QuadraticForm::from_poly` reads the term masks directly (popcount 0/1/2 toggles the constant, linear or quadratic coefficient; degree above 2 or a variable `>= n` gives `None`, as before). Test `from_poly_equals_the_anf_row_route`: equal to the `AnfRow` route on 3000 random polynomials, rejections included. Probe: `from_poly` 0.461 → 0.041 ms, engine body 0.569 → 0.062 ms. This also speeds `fes-f2` and every other caller.
- `AnfRow::from_poly` walks the set bits only (same lists, already sorted).
- `--serve-bin` (`fes_io.hpp`, host and CUDA workers): little-endian request and reply, no text. The CUDA worker uploads `[lin | quad | qsym]` from one pinned buffer in one copy, and the count comes back with the first 64 solutions in one copy and one synchronisation. `ServeWorker` speaks it; the serve test (40 systems against the host worker) covers it. On the pod, the CUDA worker’s replies equalled the host worker’s on 30 systems (459 050 solutions).

Run

One A100 80GB PCIe pod (`gvor4yci0yby96`, host `ruvct7owvq4l`, CUDA 12.8), 04:37Z–04:57Z 2026-11-14, about $0.70. ABBA per sweep: **old** = `ic` from `abecd26901` (sha256 `ad9084a9…`) with that tree's worker in text `--serve`; **new** = `ic` from `6128d645d6` (sha256 `03d96415…`) with `--serve-bin`. Same instances and seeds as `research/ic_gpu_e2e_20261113` (no mitm reference row). Raw output `raw/out_ab.tgz`; full table `RESULTS.md`.

Results

Counted ops, calls and hit rate are identical per repeat in every run of both arms; every row recovered the planted logarithm. S is the counted ops priced at the curve's calibration: pinned for the Koblitz curve (identical S everywhere), measured once per `ic bench` run for the char-2 curves (about ±0.5% between runs of the *same* arm, independent of the engine).

instance (unknowns)	target	solver ms/call old → new	Rust side ms/call old → new	whole solve s old → new
Koblitz n=23 (24)	1	1.094 → 0.580 (1.89×)	0.49 → 0.10	1.07 → 0.64 (1.68×)
	2	0.653 → 0.236 (2.77×)	0.41 → 0.11	0.78 → 0.39 (1.97×)
char-2 n=25 (26)	1	1.107 → 0.484 (2.29×)	0.60 → 0.13	3.04 → 1.61 (1.89×)
char-2 n=29 (30)	1	2.619 → 1.543 (1.70×)	1.04 → 0.18	27.35 → 17.45 (1.57×)

Largest max/min between two runs of the same arm and target: 1.20 (n=25), 1.15 (n=23), 1.004 (n=29), below every ratio above [pcc:unverified]. The first Koblitz target carries the worker start and CUDA initialisation in its round trip (new arm: 0.48 ms against 0.13 ms on the second target).

At 30 unknowns the round trip is now 1.37 ms per call against about 1.30 ms of kernel time (A100 kernel round 2), so what remains is mostly the search itself [pcc:unverified]. The CPU `fes-f2` row at n=23 moved from 13.65 to 13.44 ms per call (1.5%, outside its 0.5% same-arm spread): the `from_poly` change, in a call dominated by the search.

Against the earlier end-to-end run (different pod), the n=29 solve with `inherited-f4` took 895.5 s; with `fes-f2-gpu` it now takes 17.4 s. Counted S is unchanged and `inherited-f4` remains cheaper in counted cost at every size measured.

Not done / next

- Batching several systems per launch: the descent oracle stops at the first liftable solution, so solving the next systems ahead of time would add counted work; only worth it with a different oracle contract.
- An independent replay before these rows are cited as more than `[pcc:unverified]`.